PROJECT PLANNING AND MANAGEMENT

- Feasibility Study
    1. Technical Feasibility: The proposed system is technically feasible because all functionalities can be catered for using the specified technologies (C++ and SQ Lite). The system is also required to be basically simple but support up to 800 students, thus less complex technologies will be used. However, system performance with a large number of users represents a technical risk and may need to be considered during design and testing
    2. Operational Feasibility: Feasible because the system is hosted in the school's environment and supports the actual activities of the four user groups. However, the system must have a simple user friendly interface because teachers are not necessarily IT experts. Furthermore, the stakeholder conflicts must be resolved before implementation.
    3. Economic Feasibility: Yes, system is economically feasible because it is designed to operate on a local network without depending on internet connectivity. Since it is a relatively simple system and does not support resource intensive functionalities such as video conferencing and online payments, it greatly reduces the infrastructure and development requirements.
    4. Legal Feasibility: The system is legally feasible provided that the appropriate measures are taken to protect the students' personal information and assessment records. The information must be kept and maintained accurately, moreover access to restricted information must only be allowed to the relevant stakeholders.
    5. Schedule Feasibility: The project is considered feasible within seven weeks, although the schedule is demanding, the project is divided into defined phases each with its own challenges, thus effective time management will be necessary to complete each phase within its allocated time period.

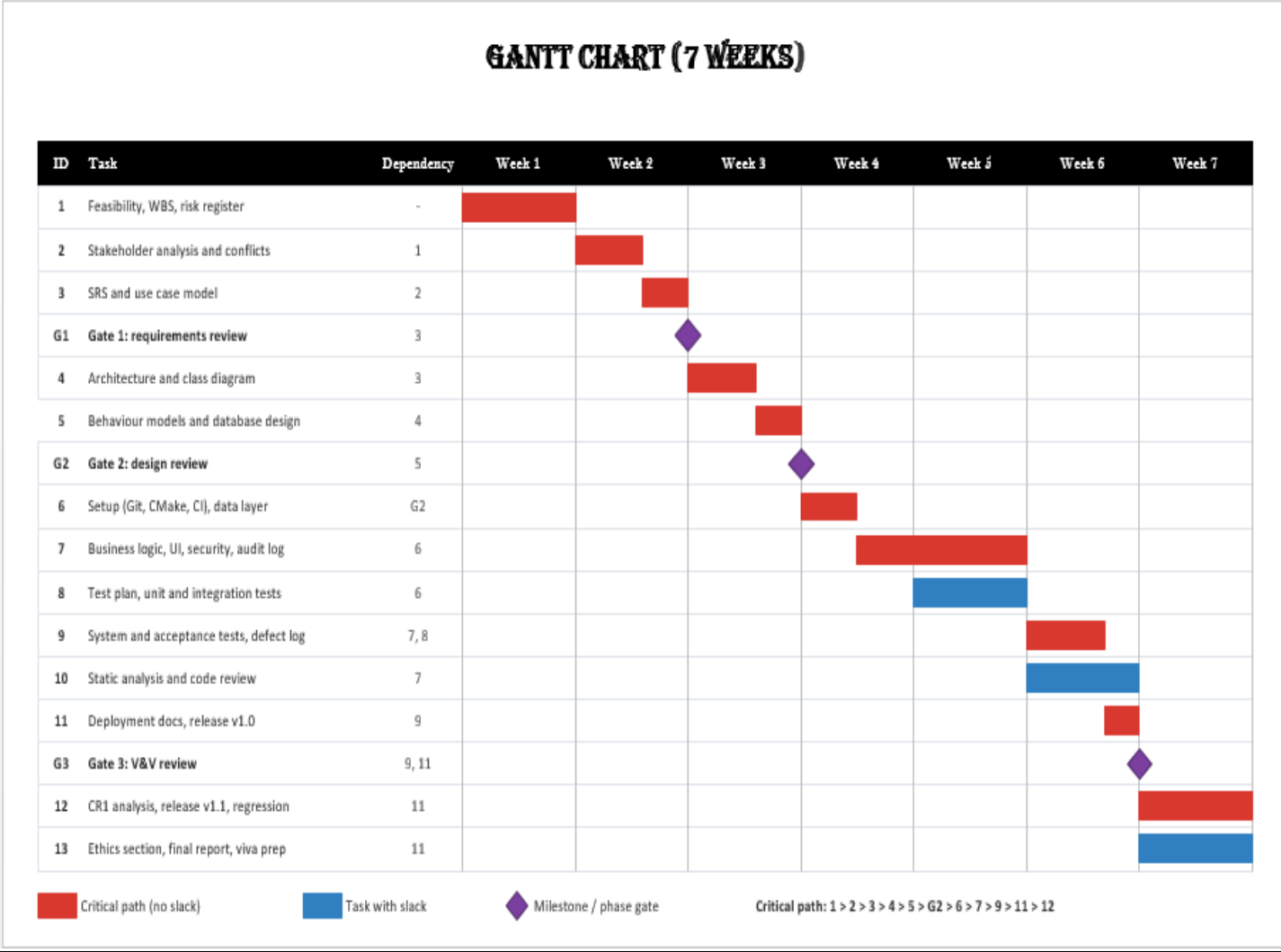
- Work Breakdown and Schedule
    a) Work Breakdown Structure

    MLMS (NAME)

    1. Project Planning And Management
        - Feasibility Study
        - Work Breakdown Structure
        - Risk register
    2. Requirements Engineering
        - Stakeholder Analysis
        - Conflict Resolution
        - Software requirements specifications
        - Use Case Model

3. System Design
    - Architecture
    - Class Diagrams and Design Patterns
    - Behavioural models
    - Database Design
4. Implementation
    - Setup: Git, CMake, CI
    - Database access layer(SQLite)
    - Business Logic and user roles
    - Security, audit log and interface
5. Verification and Validation
    - Test Strategy and Plan
    - Unit and Integration tests
    - System and acceptance tests
6. Config Management and QA
    - Git Branching and releases
    - CI Pipeline
    - Static Analysis and code review
7. Deployment
    - Deployment Diagram
    - Installation Guide
    - User Manuals and release v1.0
8. Maintenance
    - Change Request Impact Analysis
    - Implement Release v1.1
    - Regression Testing
9. Project Completion
    - Professional, legal and ethical issues
    - Final Report
    - Viva meeting

b) Gantt Chart



- Risk Management

A risk is something that might go wrong and hinder the project. A Risk Register lists each risk, scores how serious it is, with the scores changing at every phase gate as the project moves. Also says what to do about the risk and who is responsible

Risk Register: Moderated Learning Management System

ID	Risk description	Category	Likelihood (1-5)	Impact (1-5)	Exposure (L x I)	Rating	Mitigation	Owner	Status
R1	Scope creep: adding features beyond the Must requirements (or out-of-scope items such as plagiarism checks) uses up time needed for testing and documentation.	Project	4	4	16	High	Freeze the scope using MoSCoW at Gate 1. Build Must items first. Log new ideas as Could or Won't. Check the brief's out-of-scope list before adding anything.	Developer (me)	Open
R2	Falling behind schedule or illness: as the only team member, any lost week delays every later phase.	Project	3	5	15	High	Compare progress with the Gantt chart every week. Start tasks early. Keep Could items optional so they can be dropped. Keep one unplanned day per week as a buffer.	Developer (me)	Open
R3	Unresolved stakeholder conflicts: for example the Lecturer wants instant marks but the Principal wants HoD approval first, which leads to an unstable SRS and rework later.	People	4	3	12	High	List every conflict and its resolution in the stakeholder analysis. Apply the HoD-moderation rule from the case study. Get the supervisor to confirm the resolutions at Gate 1.	Developer (me)	Open
R4	Security flaw: weak password storage or SQL injection exposes learner data and marks.	Technical	2	5	10	Medium	Store salted hashes using a recognised library (e.g. libsodium with Argon2). Use parameterised queries only. Run a static analyser. Test role permissions.	Developer (me)	Open
R5	Loss of code or documents: laptop failure or accidental deletion destroys work.	Project	2	5	10	Medium	Commit to a hosted Git repository at least daily. Keep a weekly backup of documents in a second location.	Developer (me)	Open
R6	Learning curve: unfamiliar tools (C++17 features, SQLite, GoogleTest, CI) slow development.	Technical	3	3	9	Medium	Build small prototypes in Weeks 1 to 3, before the design is frozen. Use official tutorials. Choose well-documented libraries.	Developer (me)	Open
R7	Stakeholders unavailable: stakeholders or the supervisor cannot answer questions or review at the gates.	People	3	3	9	Medium	Book the gate reviews early. Use the case-study statements as the stakeholder source. Record assumptions in the SRS and confirm them at the next gate.	Supervisor	Open
R8	Poor performance with 800 students: slow login or list screens on old computers.	Technical	2	4	8	Medium	Index key columns. Test with 800 generated users. Set a measurable response target in the SRS (e.g. login within 2 seconds).	Developer (me)	Open
R9	Not enough time for testing: implementation overruns and verification is squeezed into Week 6.	Project	3	4	12	High	Write unit tests while coding each module. Set up CI in Week 4. Cut Should and Could features before cutting tests.	Developer (me)	Open
R10	Database problems: SQLite file corruption or a bad schema change causes loss of marks or audit-log data.	Technical	2	4	8	Medium	Use transactions for mark changes. Keep schema scripts in version control. Add a simple database backup step to the installation guide.	Developer (me)	Open

Gate Updates: re-score every risk at each phase gate

			Gate 1 (end of Week 2)			Gate 2 (end of Week 3)			Gate 3 (end of Week 6)			
ID	Risk (from register)	Initial exposure	L	I	Exposure	L	I	Exposure	L	I	Exposure	Notes: what changed and why
R1	Scope creep	16										
R2	Falling behind schedule or illness	15										
R3	Unresolved stakeholder conflicts	12										
R4	Security flaw	10										
R5	Loss of code or documents	10										
R6	Learning curve	9										
R7	Stakeholders unavailable	9										
R8	Poor performance with 800 students	8										
R9	Not enough time for testing	12										
R10	Database problems	8										